

Redes Neuronales (2026)

Ayudas e indicaciones para la Guía 9

Perceptrón simple y compuerta lógica OR

Reportar errores a: tristan.osan@unc.edu.ar

Índice

Introducción	1
1. Preparación del entorno	2
1.1. Bloque de código	2
1.2. Explicación breve	2
1.3. Explicación ampliada	2
2. Datos de entrada y salida de la compuerta OR	3
2.1. Tabla de verdad	3
2.2. Bloque de código	3
2.3. Explicación ampliada	3
3. Representación geométrica de OR	4
3.1. Función graficar_puntos	4
3.2. ¿Qué significa def?	4
3.3. Conversión a NumPy	5
3.4. ravel() y máscaras booleanas	5
3.5. El bucle for	5
3.6. Uso	5
4. Construcción del perceptrón simple	6
4.1. Función crear_modelo_lineal	6
4.2. torch.manual_seed	6
4.3. nn.Sequential	6
4.4. nn.Linear(2,1)	6
4.5. nn.Sigmoid	7
4.6. return	7
4.7. Creación del modelo OR	7
5. Inspección de los parámetros	7
5.1. Bloque de código	7
5.2. modelo_or[0]	7
5.3. weight y bias	7
6. Propagación hacia adelante y regla de clasificación	8
6.1. Bloque de código	8
6.2. Llamar al modelo como una función	8
6.3. torch.no_grad()	8
6.4. Umbral 0.5	9
7. Función de costo	9
7.1. Definición utilizada	9
7.2. Bloque de código	9
7.3. nn.MSELoss	9
7.4. loss_or.item()	10
8. Cálculo automático de gradientes	10
8.1. Bloque de código	10
8.2. modelo_or.zero_grad()	10
8.3. loss_or.backward()	10
8.4. Resultado típico	10
8.5. Relación con la regla de la cadena	11

9. Una actualización manual por descenso por gradiente	11
9.1. Regla matemática	11
9.2. Bloque de código	11
9.3. ¿Por qué se crea <code>modelo_manual</code> ?	12
9.4. <code>modelo_manual.parameters()</code>	12
9.5. El bucle <code>for parametro in ...</code>	12
9.6. ¿Por qué <code>torch.no_grad()</code> ?	12
9.7. Resultado de una actualización	12
10. Automatización del entrenamiento con <code>torch.optim.SGD</code>	12
10.1. Función de entrenamiento completa	12
10.2. Creación del criterio y del optimizador	13
10.3. El diccionario <code>historial</code>	13
10.4. El bucle de épocas	14
10.5. Paso 1: forward pass	14
10.6. Paso 2: costo antes de actualizar	14
10.7. Paso 3: limpiar y calcular gradientes	14
10.8. Paso 4: actualización automática	14
10.9. Paso 5: costo después de actualizar	14
11. Entrenamiento completo para OR	15
11.1. Bloque de código	15
11.2. Gráfico de los costos	15
11.3. Explicación del bloque	16
11.4. Uso	16
12. Evaluación del modelo entrenado	16
12.1. Bloque de código	16
12.2. Comparación elemento a elemento	17
12.3. Indexación de tensores	17
12.4. <i>f-strings</i>	17
12.5. Resultado final típico	17
13. Parámetros finales y frontera de decisión	18
13.1. De la sigmoide a la frontera	18
13.2. Función <code>graficar_frontera_lineal</code>	18
13.3. <code>flatten()</code> e <code>item()</code>	19
13.4. El caso de una recta vertical	19
13.5. Uso	19
14. Resumen conceptual del flujo de entrenamiento	20
15. Correspondencia entre conceptos y código	20
16. Preguntas de autoevaluación	20
17. Código completo del ejemplo OR	21
Comentarios	22

Introducción

Este tutorial explica con detalle la parte de la notebook de Google Colab dedicada a construir, entrenar y evaluar un perceptrón simple para representar la compuerta lógica OR. El texto sigue el mismo orden conceptual que el código: preparar el entorno, definir los datos, construir el modelo, realizar la propagación hacia adelante, calcular el costo, obtener gradientes, actualizar parámetros, automatizar el entrenamiento y estudiar la frontera de decisión.

La arquitectura utilizada contiene una única neurona, dos entradas y un término independiente *bias*:

$$\mathbf{x} = (x_1, x_2), \quad h = w_1x_1 + w_2x_2 + b.$$

La salida continua se obtiene aplicando una sigmoide:

$$\hat{y} = \sigma(h) = \frac{1}{1 + e^{-h}}.$$

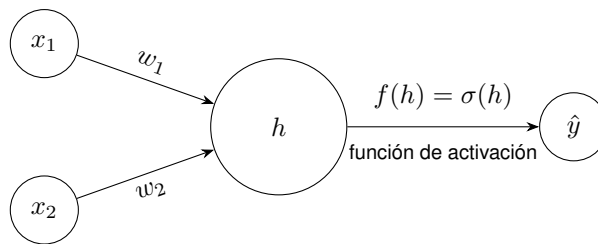


Figura 1: Perceptrón simple utilizado para representar OR. El término b se administra como `bias` de `nn.Linear`.

Objetivos de aprendizaje

- interpretar tensores, dimensiones e índices básicos de PyTorch;
- comprender qué hace `nn.Linear(2,1)`;
- distinguir los pesos `weight` del parámetro `bias`;
- interpretar `nn.Sequential` y `nn.Sigmoid`;
- ejecutar un *forward pass*;
- usar `torch.no_grad()` cuando no se requieren gradientes;
- calcular un costo con `nn.MSELoss`;
- comprender `zero_grad()`, `backward()` y los atributos `.grad`;
- realizar una actualización manual por descenso por gradiente;
- usar `torch.optim.SGD` para automatizar la actualización de parámetros;
- evaluar el modelo sobre las cuatro configuraciones de OR;
- interpretar la frontera de decisión $w_1x_1 + w_2x_2 + b = 0$.

1. Preparación del entorno

1.1. Bloque de código

```
import torch
import torch.nn as nn
import matplotlib.pyplot as plt

print("Versión de PyTorch:", torch.__version__)
torch.set_printoptions(precision=4, sci_mode=False)
```

1.2. Explicación breve

Se importan PyTorch, el submódulo de redes neuronales y Matplotlib. Después se muestra la versión de PyTorch y se modifica la forma en que los tensores se imprimen en pantalla.

1.3. Explicación ampliada

1.3.1. `import torch`

Importa la biblioteca principal de PyTorch. A partir de ese momento podemos crear tensores, realizar operaciones matemáticas y utilizar el sistema de diferenciación automática.

1.3.2. `import torch.nn as nn`

El módulo `torch.nn` contiene capas, funciones de activación, funciones de costo y la clase base utilizada para construir redes neuronales. El alias `nn` permite escribir, por ejemplo,

`nn.Linear`

en lugar de

`torch.nn.Linear.`

1.3.3. `import matplotlib.pyplot as plt`

Matplotlib se utiliza para representar los puntos de OR, la evolución del costo y la frontera de decisión. El alias habitual `plt` permite invocar funciones como `plt.plot()` o `plt.scatter()`.

1.3.4. `torch.__version__`

La expresión consulta la versión de PyTorch instalada. El valor puede cambiar entre distintas sesiones de Colab. En una ejecución utilizada para validar este tutorial se obtuvo:

```
Versión de PyTorch: 2.10.0+cpu
```

1.3.5. `torch.set_printoptions`

La instrucción

```
torch.set_printoptions(precision=4, sci_mode=False)
```

solo modifica la forma de impresión de los tensores:

- `precision=4`: muestra cuatro cifras decimales;
- `sci_mode=False`: evita, cuando es posible, la notación científica.

No modifica los valores almacenados internamente.

2. Datos de entrada y salida de la compuerta OR

2.1. Tabla de verdad

La operación lógica OR produce 1 cuando al menos una de sus entradas vale 1:

x_1	x_2	OR
0	0	0
0	1	1
1	0	1
1	1	1

2.2. Bloque de código

```
X = torch.tensor([
    [0., 0.],
    [0., 1.],
    [1., 0.],
    [1., 1.]
])

y_or = torch.tensor([
    [0.],
    [1.],
    [1.],
    [1.]
])

print("X =")
print(X)
print("\nX.shape =", X.shape)
print("y_or.shape =", y_or.shape)
```

2.3. Explicación ampliada

2.3.1. torch.tensor

Un tensor es la estructura de datos fundamental de PyTorch. En este caso, X es una matriz de cuatro filas y dos columnas:

$$X = \begin{pmatrix} 0 & 0 \\ 0 & 1 \\ 1 & 0 \\ 1 & 1 \end{pmatrix}.$$

Cada fila es un ejemplo. Cada columna representa una variable de entrada.

Los puntos decimales, por ejemplo 0. y 1., hacen que los datos sean números de punto flotante. Esto es importante porque los parámetros de una red neuronal y sus operaciones se realizan normalmente en punto flotante.

2.3.2. Forma de X

X.shape devuelve:

```
torch.Size([4, 2])
```

La primera dimensión corresponde al número de ejemplos y la segunda al número de características:

4 ejemplos $\times$ 2 entradas.

2.3.3. Forma de `y_or`

`y_or.shape` es

```
torch.Size([4, 1])
```

porque existe un valor deseado por cada uno de los cuatro ejemplos.

En esta versión no agregamos una entrada auxiliar -1 . PyTorch administra el término independiente mediante el atributo `bias` de cada capa `nn.Linear`.

3. Representación geométrica de OR

3.1. Función `graficar_puntos`

```
def graficar_puntos(X, y, titulo):
    Xn = X.detach().cpu().numpy()
    yn = y.detach().cpu().numpy().ravel()

    plt.figure(figsize=(5.5, 5))

    for clase, marcador in [(0, "o"), (1, "s")]:
        mask = yn == clase
        plt.scatter(
            Xn[mask, 0],
            Xn[mask, 1],
            s=150,
            marker=marcador,
            edgecolors="black",
            label=f"Clase {clase}"
        )

    plt.xlim(-0.3, 1.3)
    plt.ylim(-0.3, 1.3)
    plt.xticks([0, 1])
    plt.yticks([0, 1])
    plt.xlabel("x1")
    plt.ylabel("x2")
    plt.title(titulo)
    plt.grid(alpha=0.25)
    plt.legend()
    plt.show()
```

3.2. ¿Qué significa `def`?

La palabra `def` comienza la definición de una función. La cabecera

```
def graficar_puntos(X, y, titulo):
```

indica que la función recibe tres argumentos. El bloque indentado que sigue constituye el cuerpo de la función.

3.3. Conversión a NumPy

La cadena

```
X.detach().cpu().numpy()
```

realiza tres operaciones:

1. `detach()`: obtiene un tensor desligado del grafo de autograd;
2. `cpu()`: asegura que el tensor se encuentre en memoria de CPU;
3. `numpy()`: convierte el tensor a un arreglo NumPy que Matplotlib puede utilizar directamente.

En esta notebook los datos ya están en CPU y no requieren gradientes, pero escribir la secuencia completa hace que la función sea más general.

3.4. `ravel()` y máscaras booleanas

La variable `y` tiene forma $(4, 1)$. El método `ravel()` lo transforma en un vector de longitud 4. Luego:

```
mask = yn == clase
```

produce una máscara de valores `True/False`. Esa máscara permite seleccionar solamente los puntos correspondientes a una clase.

3.5. El bucle `for`

```
for clase, marcador in [(0, "o"), (1, "s")]:
```

recorre dos pares: la clase 0 se dibuja con un círculo y la clase 1 con un cuadrado.

3.6. Uso

```
graficar_puntos(X, y_or, "Compuerta OR")
```

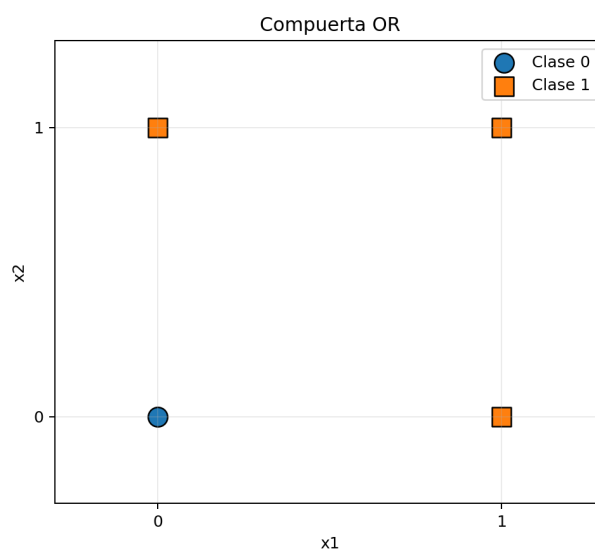


Figura 2: Las cuatro configuraciones de entrada de OR. El punto $(0, 0)$ pertenece a la clase 0 y los otros tres a la clase 1.

La figura permite anticipar una propiedad esencial: una sola recta puede separar el punto de clase 0 de los tres puntos de clase 1. Por ello OR es linealmente separable.

4. Construcción del perceptrón simple

4.1. Función `crear_modelo_lineal`

```
def crear_modelo_lineal(semilla=7):
    torch.manual_seed(semilla)

    return nn.Sequential(
        nn.Linear(2, 1),
        nn.Sigmoid()
    )
```

4.2. `torch.manual_seed`

PyTorch inicializa los pesos con valores pseudoaleatorios. La instrucción

```
torch.manual_seed(semilla)
```

fija el estado inicial del generador pseudoaleatorio. Usar la misma semilla facilita reproducir una experiencia docente. El número 7 no tiene un significado especial.

Nota. La semilla fija la inicialización; no significa que los parámetros permanezcan fijos. Durante el entrenamiento los pesos y el bias siguen modificándose.

4.3. `nn.Sequential`

`nn.Sequential` crea un modelo en el que los módulos se ejecutan en el orden en que aparecen:

$$\mathbf{x} \longrightarrow \text{Linear}(2, 1) \longrightarrow \text{Sigmoid} \longrightarrow \hat{y}.$$

Es especialmente conveniente para una arquitectura sencilla y estrictamente secuencial.

4.4. `nn.Linear(2, 1)`

Esta capa recibe dos valores y produce uno. Para cada ejemplo calcula:

$$h = w_1x_1 + w_2x_2 + b.$$

PyTorch almacena:

- los dos pesos en `weight`, con forma $(1, 2)$;
- un bias en `bias`, con forma $(1,)$.

La forma general implementada por `nn.Linear` es

$$y = \mathbf{x}A^T + b.$$

Para nuestro único nodo de salida, esto se reduce a la expresión escalar anterior.

4.5. nn.Sigmoid

La función sigmoide es

$$\sigma(h) = \frac{1}{1 + e^{-h}},$$

y produce valores entre 0 y 1. No posee parámetros aprendibles.

4.6. return

return indica qué objeto devuelve la función. Por eso:

```
modelo_or = crear_modelo_lineal(semilla=7)
```

asigna a modelo_or el objeto nn.Sequential creado dentro de la función.

4.7. Creación del modelo OR

```
modelo_or = crear_modelo_lineal(semilla=7)
print(modelo_or)
```

Una salida típica es:

```
Sequential(
  (0): Linear(in_features=2, out_features=1, bias=True)
  (1): Sigmoid()
)
```

Los números (0) y (1) son los índices de los módulos dentro de Sequential.

5. Inspección de los parámetros

5.1. Bloque de código

```
capa_lineal = modelo_or[0]

print("Pesos [w1, w2]:")
print(capa_lineal.weight)
print("Shape de weight:", capa_lineal.weight.shape)

print("\nBias b:")
print(capa_lineal.bias)
print("Shape de bias:", capa_lineal.bias.shape)
```

5.2. modelo_or[0]

El operador de indexación [0] selecciona el primer módulo de nn.Sequential. Por tanto:

```
capa_lineal = modelo_or[0]
```

no crea otra capa: crea una referencia al mismo objeto Linear que ya forma parte de modelo_or.

5.3. weight y bias

Con la semilla 7, una ejecución típica produce:

```
Pesos [w1, w2]:
Parameter containing:
tensor([[ 0.0494, -0.4260]], requires_grad=True)
Shape de weight: torch.Size([1, 2])

Bias b:
Parameter containing:
tensor([0.2252], requires_grad=True)
Shape de bias: torch.Size([1])
```

La forma (1, 2) de `weight` indica una neurona de salida conectada a dos entradas. La forma (1,) de `bias` indica un bias por neurona de salida.

El texto `requires_grad=True` señala que autograd seguirá las operaciones realizadas con ese parámetro para poder calcular gradientes.

6. Propagación hacia adelante y regla de clasificación

6.1. Bloque de código

```
with torch.no_grad():
    y_inicial_or = modelo_or(X)
    clases_iniciales_or = (y_inicial_or >= 0.5).int()

print("Salidas iniciales:")
print(y_inicial_or)

print("\nClases iniciales:")
print(clases_iniciales_or)
```

6.2. Llamar al modelo como una función

```
modelo_or(X)
```

hace que `nn.Sequential` aplique automáticamente:

$$X \rightarrow \text{Linear} \rightarrow \text{Sigmoid}.$$

Para cada fila m de X se calcula:

$$h^{(m)} = w_1 x_1^{(m)} + w_2 x_2^{(m)} + b,$$

$$\hat{y}^{(m)} = \sigma(h^{(m)}).$$

6.3. `torch.no_grad()`

El contexto

```
with torch.no_grad():
```

desactiva temporalmente el seguimiento de gradientes. Se usa aquí porque solo queremos observar las salidas iniciales; no vamos a ejecutar `backward()` a partir de estas operaciones.

Nota. `torch.no_grad()` no borra gradientes ya existentes. Su función es evitar que las nuevas operaciones se registren para diferenciación automática.

6.4. Umbral 0.5

Como

$$\sigma(0) = 0.5$$

y la sigmoide es creciente,

$$\hat{y} \geq 0.5 \iff h \geq 0.$$

Por ello usamos:

```
(y_inicial_or >= 0.5).int()
```

La comparación produce valores booleanos. El método `.int()` los convierte en enteros 0 y 1.

Con la inicialización de la semilla 7 se obtiene aproximadamente:

```
Salidas iniciales:
tensor([[0.5561],
        [0.4500],
        [0.5682],
        [0.4622]])

Clases iniciales:
tensor([[1],
        [0],
        [1],
        [0]], dtype=torch.int32)
```

El modelo todavía no fue entrenado, por lo que no esperamos que estas clases coincidan con OR.

7. Función de costo

7.1. Definición utilizada

La notebook emplea, por motivos didácticos,

$$E = \frac{1}{2M} \sum_{m=1}^M \left(\hat{y}^{(m)} - y^{(m)} \right)^2.$$

El factor $1/2$ simplifica la derivada del término cuadrático y no modifica la posición del mínimo.

7.2. Bloque de código

```
criterio = nn.MSELoss()

y_pred = modelo_or(X)
loss_or = 0.5 * criterio(y_pred, y_or)

print("Costo inicial para OR:", loss_or.item())
```

7.3. `nn.MSELoss`

Con su configuración predeterminada, `nn.MSELoss()` calcula el promedio de los errores cuadrados:

$$\text{MSE} = \frac{1}{M} \sum_m (\hat{y}^{(m)} - y^{(m)})^2.$$

Multiplicar por 0.5 implementa exactamente la convención elegida en el tutorial.

7.4. `loss_or.item()`

`loss_or` es un tensor escalar. El método `.item()` extrae el único valor y lo convierte en un número ordinario de Python, útil para imprimir o guardar en una lista.

Con la semilla 7 el costo inicial es aproximadamente:

```
Costo inicial para OR: 0.13592258095741272
```

8. Cálculo automático de gradientes

8.1. Bloque de código

```
modelo_or.zero_grad()

y_pred = modelo_or(X)
loss_or = 0.5 * criterio(y_pred, y_or)
loss_or.backward()

print("Gradientes de los pesos:")
print(modelo_or[0].weight.grad)

print("\nGradiente del bias:")
print(modelo_or[0].bias.grad)
```

8.2. `modelo_or.zero_grad()`

PyTorch acumula gradientes por defecto. `zero_grad()` limpia los gradientes almacenados antes de calcular los correspondientes a una nueva iteración.

Conceptualmente:

gradientes viejos $\rightarrow 0 \rightarrow$ nuevo backward.

8.3. `loss_or.backward()`

Durante el forward pass, PyTorch construye un grafo de operaciones. `backward()` aplica la regla de la cadena a ese grafo y calcula:

$$\frac{\partial E}{\partial w_1}, \quad \frac{\partial E}{\partial w_2}, \quad \frac{\partial E}{\partial b}.$$

Los gradientes se almacenan en:

```
modelo_or[0].weight.grad
modelo_or[0].bias.grad
```

8.4. Resultado típico

```
Gradientes de los pesos:
tensor([[ -0.0599, -0.0675]])

Gradiente del bias:
tensor([ -0.0596])
```

8.5. Relación con la regla de la cadena

Para un ejemplo m ,

$$E^{(m)} = \frac{1}{2} \left(\hat{y}^{(m)} - y^{(m)} \right)^2,$$

$$\hat{y}^{(m)} = \sigma \left(h^{(m)} \right),$$

$$h^{(m)} = w_1 x_1^{(m)} + w_2 x_2^{(m)} + b.$$

Entonces, para un peso w_i ,

$$\frac{\partial E^{(m)}}{\partial w_i} = \frac{\partial E^{(m)}}{\partial \hat{y}^{(m)}} \frac{\partial \hat{y}^{(m)}}{\partial h^{(m)}} \frac{\partial h^{(m)}}{\partial w_i}.$$

Como

$$\frac{\partial h^{(m)}}{\partial b} = 1,$$

el bias recibe su propio gradiente. La llamada a `backward()` automatiza precisamente estas derivadas.

`backward()` calcula gradientes, pero no actualiza los parámetros. La actualización es una operación separada.

9. Una actualización manual por descenso por gradiente

9.1. Regla matemática

Si agrupamos los parámetros como

$$\theta = (w_1, w_2, b),$$

una actualización es:

$$\theta^{(k+1)} = \theta^{(k)} - \eta \nabla E \left(\theta^{(k)} \right).$$

Para cada parámetro:

$$\theta_i \leftarrow \theta_i - \eta \frac{\partial E}{\partial \theta_i}.$$

9.2. Bloque de código

```
eta = 0.5

modelo_manual = crear_modelo_lineal(semilla=7)
criterio_manual = nn.MSELoss()

# Forward y backward
y_pred = modelo_manual(X)
loss_antes_manual = 0.5 * criterio_manual(y_pred, y_or)

modelo_manual.zero_grad()
loss_antes_manual.backward()

print("Costo antes:", loss_antes_manual.item())

with torch.no_grad():
    for parametro in modelo_manual.parameters():
        parametro -= eta * parametro.grad

with torch.no_grad():
```

```
loss_despues_manual = 0.5 * criterio_manual(modelo_manual(X), y_or)

print("Costo después:", loss_despues_manual.item())
```

9.3. ¿Por qué se crea `modelo_manual`?

Se crea un modelo nuevo para aislar la demostración de una única actualización. Así no se modifica accidentalmente el modelo que utilizaremos después para el entrenamiento completo.

9.4. `modelo_manual.parameters()`

El método devuelve un iterable con todos los parámetros aprendibles del modelo. En este caso incluye:

- la matriz `weight`;
- el vector `bias`.

9.5. El bucle `for parametro in ...`

El mismo código de actualización puede aplicarse a todos los parámetros sin escribir una fórmula distinta para cada uno:

```
for parametro in modelo_manual.parameters():
    parametro -= eta * parametro.grad
```

La expresión `-=` es una asignación abreviada. Conceptualmente:

```
parametro = parametro - eta * parametro.grad
```

9.6. ¿Por qué `torch.no_grad()`?

La actualización de parámetros pertenece al algoritmo de optimización. No queremos que autograd construya un grafo para esa operación. Por eso la realizamos dentro de:

```
with torch.no_grad():
```

9.7. Resultado de una actualización

Con los valores utilizados en la notebook:

```
Costo antes: 0.13592258095741272
Costo después: 0.1301986575126648
```

En este paso concreto el costo disminuye. Esto ilustra el objetivo del descenso por gradiente: mover los parámetros en una dirección que reduzca la función costo.

10. Automatización del entrenamiento con `torch.optim.SGD`

10.1. Función de entrenamiento completa

```
def entrenar(modelo, X, y, epocas=4000, eta=0.5):
    criterio = nn.MSELoss()
    optimizador = torch.optim.SGD(modelo.parameters(), lr=eta)

    historial = {
        "antes": [],
        "despues": []
    }

    for _ in range(epocas):
        # 1. Forward pass
        y_pred = modelo(X)

        # 2. Costo antes de actualizar
        loss_antes = 0.5 * criterio(y_pred, y)
        historial["antes"].append(loss_antes.item())

        # 3. Gradientes
        optimizador.zero_grad()
        loss_antes.backward()

        # 4. Actualización de parámetros
        optimizador.step()

        # 5. Costo después de actualizar
        with torch.no_grad():
            y_pred_despues = modelo(X)
            loss_despues = 0.5 * criterio(y_pred_despues, y)

        historial["despues"].append(loss_despues.item())

    return historial
```

10.2. Creación del criterio y del optimizador

```
criterio = nn.MSELoss()
optimizador = torch.optim.SGD(modelo.parameters(), lr=eta)
```

`torch.optim.SGD` implementa descenso por gradiente estocástico como clase optimizadora. En esta notebook, sin embargo, cada actualización usa simultáneamente los cuatro ejemplos de OR. Por eso, operacionalmente, estamos utilizando *full-batch gradient descent*: el objeto se llama SGD, pero el lote entregado en cada paso contiene todo el conjunto.

El argumento `lr=eta` fija la tasa de aprendizaje:

$$\eta = 0.5.$$

10.3. El diccionario `historial`

```
historial = {
    "antes": [],
    "despues": []
}
```

Un diccionario asocia claves con valores. Aquí cada clave apunta a una lista inicialmente vacía. Se guardarán dos series del mismo costo:

- "antes": costo con los parámetros actuales;
- "despues": costo después de aplicar la actualización.

10.4. El bucle de épocas

```
for _ in range(epocas):
```

`range(epocas)` genera una secuencia de índices. Se usa el nombre `_` porque el valor del índice no se necesita dentro del cuerpo del bucle.

10.5. Paso 1: forward pass

```
y_pred = modelo(X)
```

Calcula simultáneamente las predicciones para los cuatro ejemplos usando los parámetros de la época actual.

10.6. Paso 2: costo antes de actualizar

```
loss_antes = 0.5 * criterio(y_pred, y)
historial["antes"].append(loss_antes.item())
```

`append()` agrega un elemento al final de una lista. Guardamos un número de Python mediante `item()` y no el tensor completo.

10.7. Paso 3: limpiar y calcular gradientes

```
optimizador.zero_grad()
loss_antes.backward()
```

`optimizador.zero_grad()` borra los gradientes de los parámetros administrados por el optimizador. Cumple aquí el mismo propósito práctico que `modelo.zero_grad()`.

Luego `backward()` calcula los nuevos gradientes con los parámetros actuales.

10.8. Paso 4: actualización automática

```
optimizador.step()
```

`step()` recorre los parámetros administrados por el optimizador y aplica la regla de actualización de SGD. Conceptualmente:

$$\theta_i \leftarrow \theta_i - \eta \frac{\partial E}{\partial \theta_i}.$$

Esta línea sustituye el bucle manual sobre `modelo.parameters()`.

10.9. Paso 5: costo después de actualizar

```
with torch.no_grad():
    y_pred_despues = modelo(X)
    loss_despues = 0.5 * criterio(y_pred_despues, y)
```

Ahora el modelo ya contiene los nuevos parámetros. Como esta segunda evaluación solo se registra para seguimiento, se realiza sin construir otro grafo.

Si llamamos $\theta^{(k)}$ a los parámetros antes de actualizar,

$$E_{\text{antes}}^{(k)} = E\left(\theta^{(k)}\right),$$

y después de `step()`:

$$\theta^{(k+1)} = \theta^{(k)} - \eta \nabla E\left(\theta^{(k)}\right),$$

$$E_{\text{después}}^{(k)} = E\left(\theta^{(k+1)}\right).$$

Como usamos todo el conjunto de entrenamiento en cada paso y no hay muestreo aleatorio durante las épocas, salvo redondeo numérico:

$$E_{\text{después}}^{(k)} \approx E_{\text{antes}}^{(k+1)}.$$

11. Entrenamiento completo para OR

11.1. Bloque de código

```
modelo_or = crear_modelo_lineal(semilla=7)
historial_or = entrenar(
    modelo_or,
    X,
    y_or,
    epocas=4000,
    eta=0.5
)
```

Se crea un modelo nuevo, con una inicialización conocida, y se entrena durante 4000 épocas. La función `entrenar` modifica `modelo_or` en el lugar; al finalizar, el objeto contiene los parámetros aprendidos.

11.2. Gráfico de los costos

```
def graficar_costos_antes_despues(historial, titulo):
    epocas = range(1, len(historial["antes"]) + 1)

    plt.figure(figsize=(7.5, 4.5))
    plt.plot(epocas, historial["antes"], label="Antes de actualizar")
    plt.plot(
        epocas,
        historial["despues"],
        "--",
        label="Después de actualizar"
    )
    plt.xlabel("Época")
    plt.ylabel("Costo")
    plt.title(titulo)
    plt.grid(alpha=0.25)
    plt.legend()
    plt.show()
```

11.3. Explicación del bloque

`len(historial["antes"])` devuelve el número de épocas registradas. `range(1,...)` crea las etiquetas horizontales comenzando en 1. Las dos llamadas a `plt.plot` representan las dos evaluaciones temporales del mismo costo.

La segunda curva usa `"-"` para dibujarse con línea discontinua. `label` asigna el texto que luego aparece en `plt.legend()`.

11.4. Uso

```
graficar_costos_antes_despues(
    historial_or,
    "OR: costo antes y después de cada actualización"
)
```

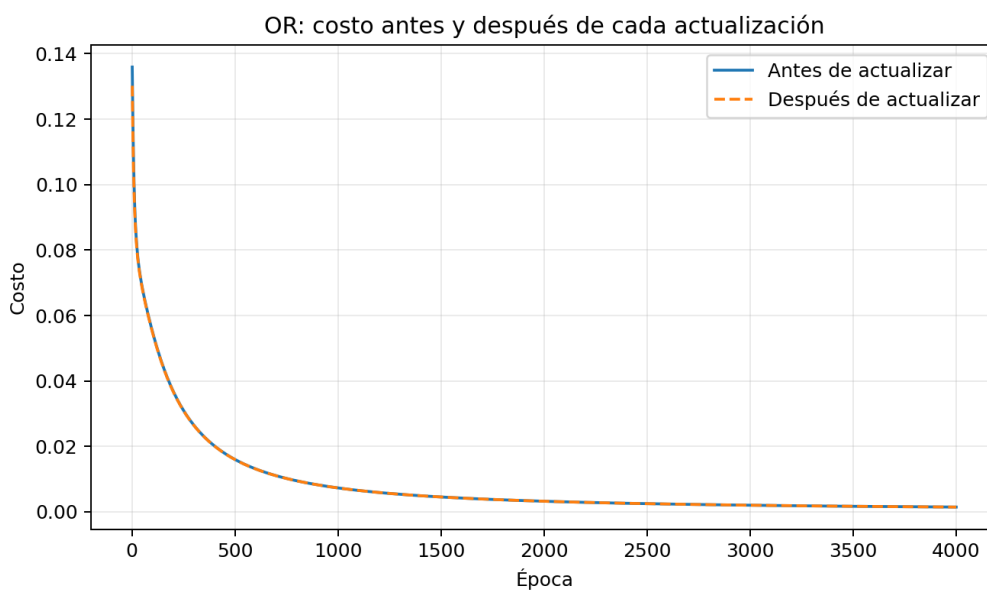


Figura 3: Evolución del costo durante el entrenamiento de OR. Las dos curvas representan la misma función evaluada inmediatamente antes y después de cada actualización.

La reducción sostenida del costo indica que las actualizaciones están moviendo los parámetros hacia una solución adecuada para OR.

12. Evaluación del modelo entrenado

12.1. Bloque de código

```
with torch.no_grad():
    salidas_or = modelo_or(X)
    pred_or = (salidas_or >= 0.5).int()

aciertos_or = (pred_or == y_or.int()).sum().item()

print(" x1   x2   salida   predicción   correcta")
print("-----")
```

```

for i in range(len(X)):
    print(
        f" {int(X[i,0])}    {int(X[i,1])}    "
        f"{salidas_or[i].item():.4f}          "
        f"{pred_or[i].item()}                "
        f"{int(y_or[i].item())}"
    )

print(f"\nAciertos: {aciertos_or}/{len(X)}")
print("Costo final:", historial_or["despues"][-1])

```

12.2. Comparación elemento a elemento

```
pred_or == y_or.int()
```

produce un tensor booleano. `sum()` cuenta cuántos elementos son verdaderos y `item()` extrae el resultado numérico.

12.3. Indexación de tensores

`X[i,0]` significa: fila i , columna 0. Por tanto corresponde a x_1 del ejemplo i . De modo análogo, `X[i,1]` corresponde a x_2 .

12.4. *f-strings*

Una cadena precedida por `f` permite insertar expresiones entre llaves:

```
f"Aciertos: {aciertos_or}/{len(X)}"
```

La expresión

```
{salidas_or[i].item():.4f}
```

indica que el número se imprimirá con cuatro decimales.

12.5. Resultado final típico

Después de 4000 épocas, con la semilla e hiperparámetros anteriores:

x1	x2	salida	predicción	correcta
0	0	0.0813	0	0
0	1	0.9495	1	1
1	0	0.9495	1	1
1	1	0.9997	1	1

Aciertos: 4/4
Costo final: 0.0014637877466157079

Los valores sigmoideos no son exactamente 0 y 1. Para clasificar, lo importante es de qué lado del umbral 0.5 se encuentran.

13. Parámetros finales y frontera de decisión

En una ejecución validada se obtuvieron aproximadamente:

$$w_1 = 5.3590, \quad w_2 = 5.3588, \quad b = -2.4242.$$

La preactivación aprendida es entonces aproximadamente:

$$h = 5.3590x_1 + 5.3588x_2 - 2.4242.$$

13.1. De la sigmoide a la frontera

La regla de clasificación es:

$$\hat{y} \geq 0.5 \iff h \geq 0.$$

La frontera es el conjunto de puntos para los cuales $h = 0$:

$$w_1x_1 + w_2x_2 + b = 0.$$

Si $w_2 \neq 0$,

$$x_2 = -\frac{w_1x_1 + b}{w_2}.$$

Por eso una sola neurona con dos entradas induce una frontera lineal en el plano (x_1, x_2) .

13.2. Función graficar_frontera_lineal

```
def graficar_frontera_lineal(modelo, X, y, titulo):
    Xn = X.detach().cpu().numpy()
    yn = y.detach().cpu().numpy().ravel()

    w = modelo[0].weight.detach().flatten()
    b = modelo[0].bias.detach().item()

    plt.figure(figsize=(5.5, 5))

    for clase, marcador in [(0, "o"), (1, "s")]:
        mask = yn == clase
        plt.scatter(
            Xn[mask, 0], Xn[mask, 1],
            s=150, marker=marcador,
            edgecolors="black", label=f"Clase {clase}"
        )

    if abs(w[1].item()) > 1e-10:
        x1 = torch.linspace(-0.3, 1.3, 200)
        x2 = -(w[0] * x1 + b) / w[1]
        plt.plot(x1.numpy(), x2.numpy(), label="Frontera h=0")
    elif abs(w[0].item()) > 1e-10:
        x1_const = -b / w[0].item()
        plt.axvline(x1_const, label="Frontera h=0")

    plt.xlim(-0.3, 1.3)
    plt.ylim(-0.3, 1.3)
    plt.xticks([0, 1])
    plt.yticks([0, 1])
    plt.xlabel("x1")
    plt.ylabel("x2")
    plt.title(titulo)
```

```
plt.grid(alpha=0.25)
plt.legend()
plt.show()
```

13.3. `flatten()` e `item()`

`weight` tiene forma $(1, 2)$. `flatten()` lo convierte en un vector de dos elementos para acceder cómodamente a w_1 y w_2 . El `bias` tiene un único elemento y `item()` lo convierte a un escalar de Python.

13.4. El caso de una recta vertical

La fórmula

$$x_2 = -\frac{w_1 x_1 + b}{w_2}$$

no puede utilizarse si $w_2 = 0$. Por eso el código comprueba primero que $|w_2|$ no sea demasiado pequeño. Si lo fuera, pero $w_1 \neq 0$, la frontera sería vertical:

$$x_1 = -\frac{b}{w_1}.$$

13.5. Uso

```
graficar_frontera_lineal(
    modelo_or,
    X,
    y_or,
    "OR: frontera de decisión aprendida"
)
```

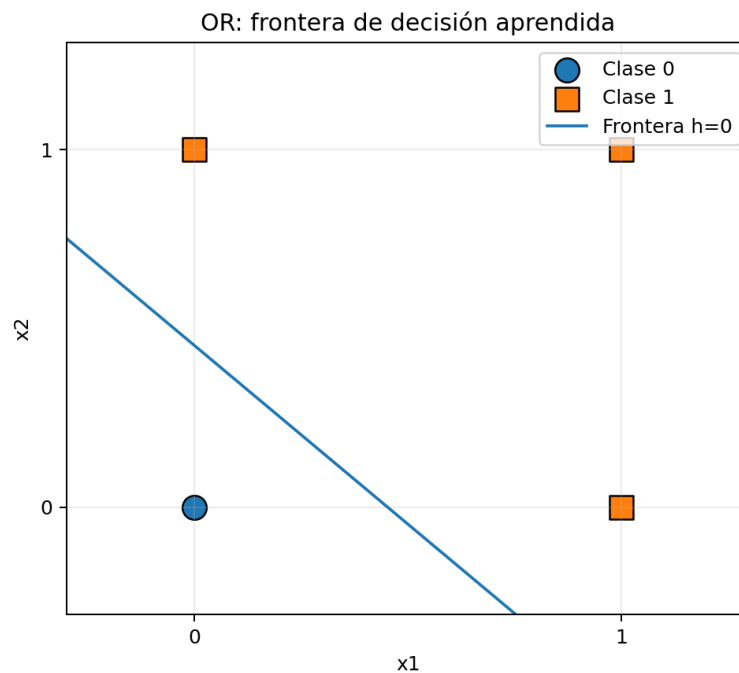
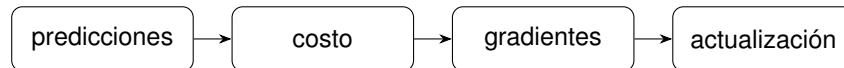


Figura 4: Frontera lineal aprendida para OR. El punto $(0, 0)$ queda de un lado y los tres puntos de clase 1 del otro.

La figura confirma geoméricamente por qué esta arquitectura puede representar OR.

14. Resumen conceptual del flujo de entrenamiento

Todo el proceso puede resumirse como:



Para cada época:

1. **Predicciones:** `y_pred = modelo(X)`.
2. **Costo:** se compara `y_pred` con `y`.
3. **Gradientes:** `loss.backward()`.
4. **Actualización:** `optimizador.step()`.
5. Se vuelve a comenzar con los nuevos parámetros.

La operación OR puede aprenderse porque existe al menos una combinación de w_1 , w_2 y b cuya frontera lineal separa correctamente las dos clases.

15. Correspondencia entre conceptos y código

Concepto	Código
Datos de entrada	<code>X</code>
Salida deseada	<code>y_or</code>
Pesos	<code>modelo_or[0].weight</code>
Bias	<code>modelo_or[0].bias</code>
Preactivación	cálculo interno de <code>nn.Linear(2,1)</code>
Activación sigmoide	<code>nn.Sigmoid()</code>
Forward pass	<code>modelo_or(X)</code>
Costo	<code>0.5 * nn.MSELoss()(y_pred, y_or)</code>
Limpiar gradientes	<code>zero_grad()</code>
Calcular gradientes	<code>loss.backward()</code>
Gradientes de pesos	<code>weight.grad</code>
Gradiente del bias	<code>bias.grad</code>
Actualizar parámetros	<code>optimizador.step()</code>
Evaluar sin autograd	<code>with torch.no_grad():</code>
Extraer un escalar	<code>tensor.item()</code>
Frontera de decisión	$w_1x_1 + w_2x_2 + b = 0$

16. Preguntas de autoevaluación

1. ¿Qué significan las dos dimensiones de `X.shape == (4,2)`?
2. ¿Por qué `nn.Linear(2,1)` tiene dos pesos pero solo un bias?
3. ¿Qué diferencia hay entre `modelo_or[0].weight` y `modelo_or[0].weight.grad`?
4. ¿Por qué `loss.backward()` no actualiza los parámetros?
5. ¿Para qué se usa `zero_grad()`?
6. ¿Para qué se usa `torch.no_grad()`?

7. ¿Por qué multiplicamos `nn.MSELoss()` por 0.5 en esta guía?
8. ¿Qué representa `loss.item()`?
9. ¿Qué hace `optimizador.step()`?
10. ¿Por qué la frontera de decisión corresponde a $h = 0$?
11. ¿Por qué OR puede separarse con una sola recta?
12. ¿Qué diferencia existe entre una salida sigmoidea continua y la clase obtenida con umbral 0.5?

17. Código completo del ejemplo OR

El siguiente bloque reúne los elementos esenciales del ejemplo en una única pieza de código.

```
import torch
import torch.nn as nn
import matplotlib.pyplot as plt

torch.set_printoptions(precision=4, sci_mode=False)

X = torch.tensor([
    [0., 0.],
    [0., 1.],
    [1., 0.],
    [1., 1.]
])

y_or = torch.tensor([
    [0.],
    [1.],
    [1.],
    [1.]
])

def crear_modelo_lineal(semilla=7):
    torch.manual_seed(semilla)
    return nn.Sequential(
        nn.Linear(2, 1),
        nn.Sigmoid()
    )

def entrenar(modelo, X, y, epocas=4000, eta=0.5):
    criterio = nn.MSELoss()
    optimizador = torch.optim.SGD(modelo.parameters(), lr=eta)

    historial = {"antes": [], "despues": []}

    for _ in range(epocas):
        y_pred = modelo(X)
        loss_antes = 0.5 * criterio(y_pred, y)
        historial["antes"].append(loss_antes.item())

        optimizador.zero_grad()
        loss_antes.backward()
        optimizador.step()
```

```

        with torch.no_grad():
            y_pred_despues = modelo(X)
            loss_despues = 0.5 * criterio(y_pred_despues, y)

        historial["despues"].append(loss_despues.item())

    return historial

modelo_or = crear_modelo_lineal(semilla=7)
historial_or = entrenar(
    modelo_or,
    X,
    y_or,
    epocas=4000,
    eta=0.5
)

with torch.no_grad():
    salidas_or = modelo_or(X)
    pred_or = (salidas_or >= 0.5).int()

aciertos_or = (pred_or == y_or.int()).sum().item()

print("Salidas:")
print(salidas_or)
print("\nClases:")
print(pred_or)
print(f"\nAciertos: {aciertos_or}/{len(X)}")
print("Costo final:", historial_or["despues"][-1])

```

Comentarios

El ejemplo OR permite estudiar el ciclo completo de aprendizaje de una neurona sin introducir todavía la dificultad geométrica de XOR. La capa `nn.Linear(2,1)` implementa la preactivación $h = w_1x_1 + w_2x_2 + b$, la sigmoide transforma esa preactivación en una salida continua, el costo cuantifica el error, autograd calcula los gradientes y SGD actualiza los parámetros. El resultado final puede interpretarse tanto numéricamente, mediante las cuatro predicciones, como geométricamente, mediante la frontera lineal $h = 0$.

Esta secuencia proporciona la base directa para repetir el mismo procedimiento con XOR y analizar posteriormente qué ocurre cuando el problema deja de ser linealmente separable.